Software Dependability

Software Fault Avoidance in Specification and Implementation

A Single Line of Code Can Disrupt Worldwide Communication

Software cannot be seen, felt, or touched, but it is responsible for most functions and failures in critical systems.

A single line of code can crash an entire telephone network.

On January 15th, 1990, at 2:25 p.m., AT&T's network center in New Jersey experienced a surge of red alerts as rapidly spreading bugs affected its worldwide switches. Usual fixes did not resolve the issue, and for nine hours, nearly half of all calls failed to connect.
By 11:30 pm, traffic had subsided, and the system had stabilized, but AT&T had lost over $60 million, with additional losses expected for businesses relying on their phones. The failure undermined AT&T’s reputation for reliability.

What went wrong?

https://users.csc.calpoly.edu/~jdalbey/SWE/Papers/att_collapse

A Single Line of Code Can Disrupt Worldwide Communication

A routine self-test on a New York City phone switch triggered a four-second reset, during which messages queued up.

When the switch rebooted, it sent two closely timed messages to another switch. Because of a software bug, this switch overwrote critical data, causing the receiving switch and its backup to shut down and restart, allowing the same error to spread across 114 switches nationwide.

The cascading failures disabled the network for nine hours, blocking over 50 million calls.

https://users.csc.calpoly.edu/~jdalbey/SWE/Papers/att_collapse

A Single Line of Code Can Disrupt Worldwide Communication

```
1  while (ring receive buffer not empty
          and side buffer not empty) DO
2     Initialize pointer to first message in side buffer
      or ring receive buffer
3     get copy of buffer
4     switch (message)
5        case (incoming_message):
6              if (sending switch is out of service) DO
7                  if (ring write buffer is empty) DO
8                      send "in service" to status map
9                  else
10                     break
                   END IF
11             process incoming message, set up pointers to
               optional parameters
12             break
         END SWITCH
13    do optional parameter work
```

When two near-simultaneous messages arrived, the destination switch—busy with the first (buffer not empty, **line 7**)—should have stayed in the `if` path, processed the second message, and set the database pointers (**line 11**). Instead, the `break` (**line 10)** exited the entire `case`, diverting flow into optional parameter handling that overwrote critical data (**line 13**). Error correction detected the corruption and forced a reset. Since every switch ran the same software, these resets cascaded across the network, ultimately incapacitating the system.

https://users.csc.calpoly.edu/~jdalbey/SWE/Papers/att_collapse
https://www.youtube.com/watch?v=qCEQP8Fp7PY

The Fundamental Principle of Dependability

“If all faults to which a system is subject are identified and their effects are properly mitigated, the system will meet its dependability requirements.”

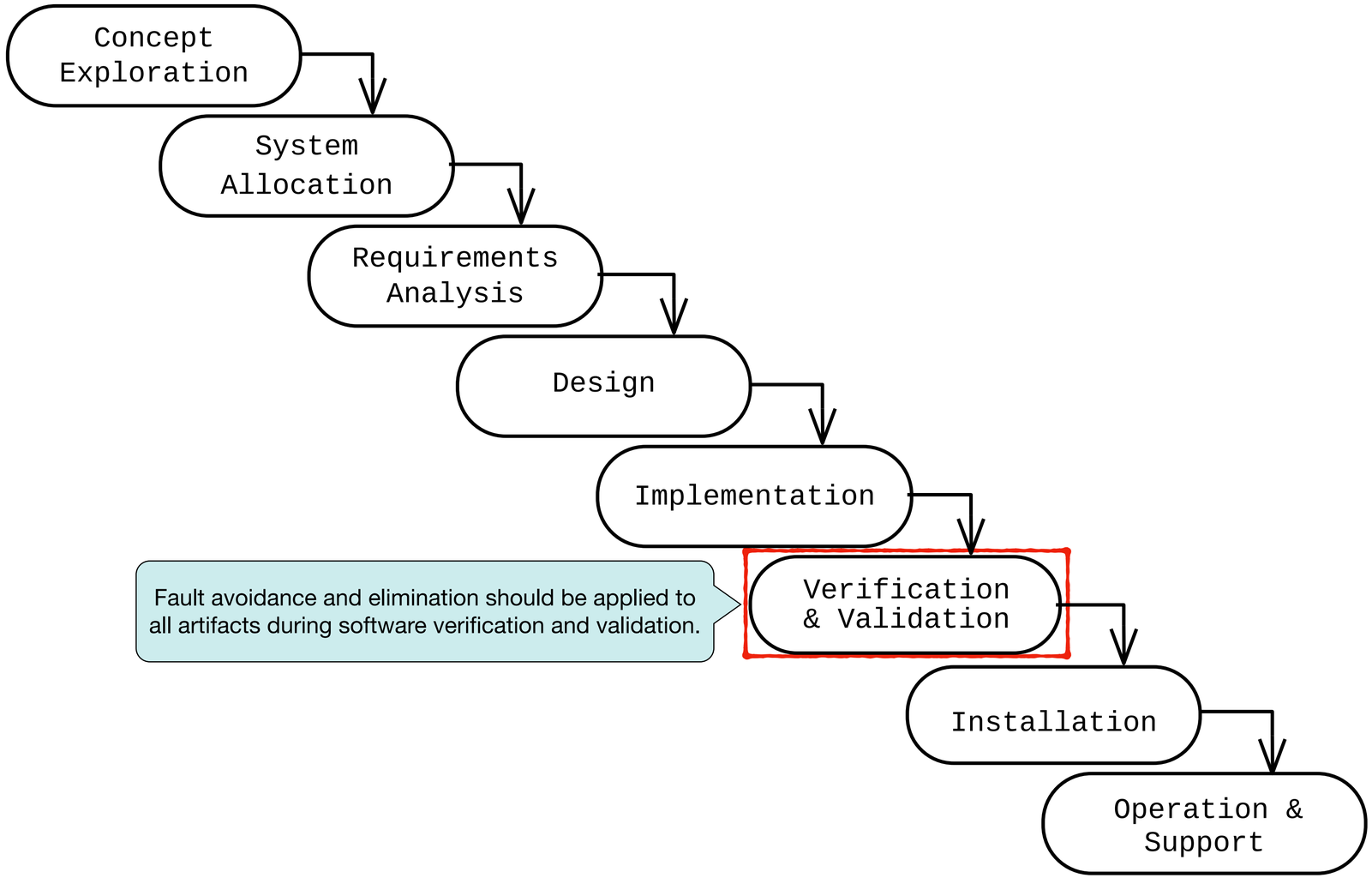
There are different ways to mitigate faults; they can be

- **Avoided**: If we build systems without faults, we do not need to do anything else.
- **Eliminated**: Do you remember software testing?
- **Tolerated**: Managed in a way that they do not cause failures.
- **Forecast**: Suppose when their effects will occur and be prepared for them.

Since not all projects are the same and perfection is not possible, we need to address dependability issues.

This course primarily focuses on fault avoidance and elimination, as fault tolerance and forecasting are more challenging to apply in software. This week, we learn how to avoid faults!

Verification and Validation



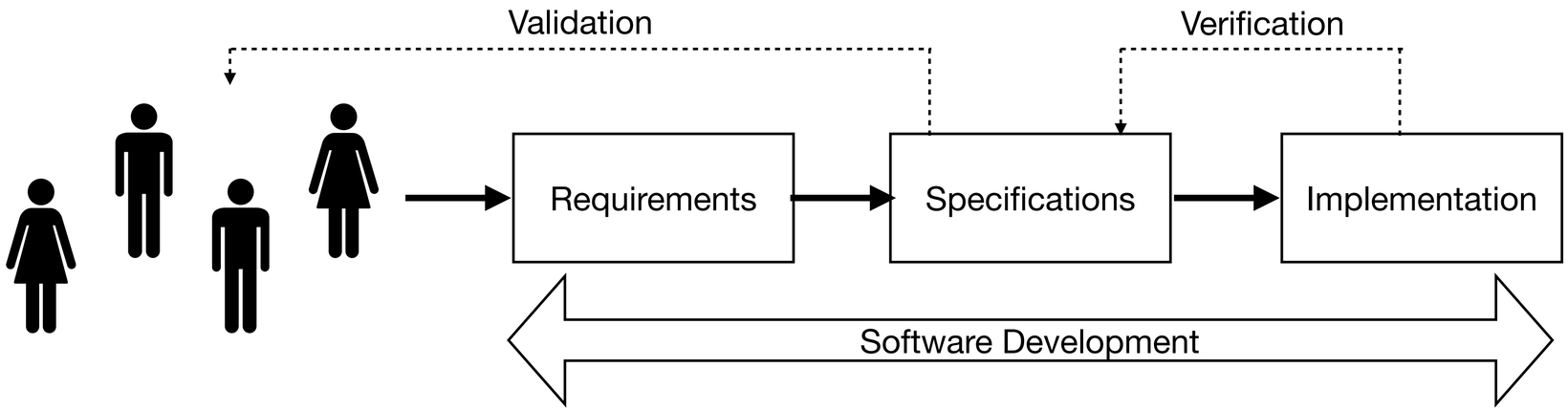
Verification and Validation

Did we build the right thing?

Validation. Developing the necessary assurance that the documented specification addresses the problem stated in the abstract requirements, i.e., the system will solve the problem posed by the user.

Verification. Showing that one software representation refines another, typically showing that the executable software accurately implements the documented specification.

Did we build the thing right?



Verification and Validation

Validation Errors. Something that the user did not want but was implemented correctly:
- erroneous specifications
- correct implementation

Verification Errors. Something that the user wanted but was implemented incorrectly:
- correct specifications
- erroneous implementation

Validation enables fault avoidance, whereas **Verification** enables fault removal.

Natural Language is Difficult

Natural language is great for everyday communication, but it is problematic in software engineering for several reasons:

- **Ambiguity**: Natural language is inherently ambiguous, e.g., ”Ibis redibis non morieris in bello”
- **Imprecision**: Many details are assumed to be obvious and are communicated through general communication.
- **Informality**: There is no method to automatically verify its syntax or other properties.
- **Dialects**: The same language varies across different countries or regions.

Writing Specifications is Difficult

Most defects are introduced during the specification process. Several issues can arise:

- **Incorrect functionality**: The stated functionality does not accurately describe what the system is supposed to do.
- **Missing functionality**: The stated functionality omits a feature that the system should perform.
- **Incompleteness**: Some inputs or conditions are not specified for the functionality.
- **Ambiguity**: The stated functionality is open to multiple interpretations.
- **Inconsistency**: Part of a stated functionality conflicts with another.
- **Naïveté**: The specification demands a desired system that cannot be built because of technical limitations.

Formal Languages

A formal language is any language with a formal syntax and a formal semantics.
A formal syntax ensures there is no doubt about what constitutes a valid statement in the language.
A formal semantics explains how the different constructs in the language manipulate the symbols used.

Any programming language qualifies as a formal language.
However, aside from these, many software engineers do not master formal languages.

Benefits of Formal Languages

Formal languages offer several advantages:

- **Better understanding**: Formal languages are much more precise than natural language.
- **Higher accuracy and completeness**: Their strict structure results in more accurate and complete artifacts.
- **Syntax validation**: The syntax can be automatically verified using a program similar to a compiler.
- **Type checking**: Types are a fundamental component of any formal language and can be automatically verified.
- **Putative or speculative theorems**: Since the specification is a set of axioms, the proposed theorem is expected to be true. If the implementation correctly follows the specifications, it will demonstrate the same properties.

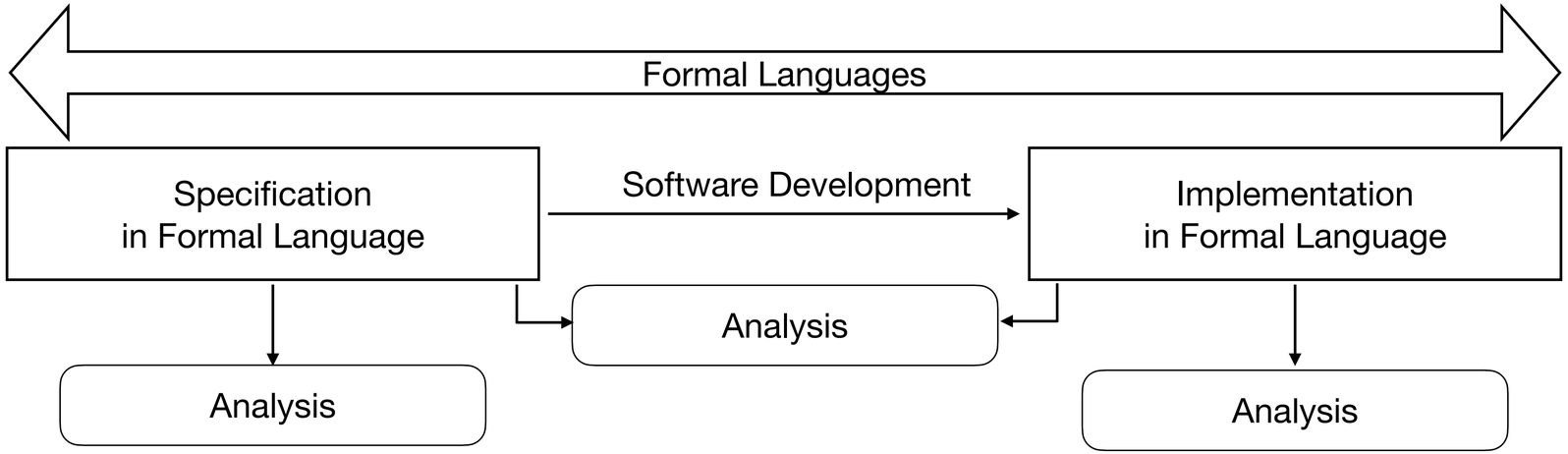
Formal Methods

Formal methods are mathematically based techniques for *specifying*, *developing*, and *verifying* software and hardware.

Their primary goal is to enhance the reliability of these systems, particularly in areas where failure could have catastrophic consequences, such as aerospace, medical devices, nuclear systems, and critical infrastructure.

Formal languages help reduce reliance on natural language during requirements analysis and software design.

Formal verification uses specifications written in a formal language and the implementation to prove that the implementation refines the specifications (*partial correctness*). If the program is expected to terminate, a proof of termination can be added to establish *total correctness*.

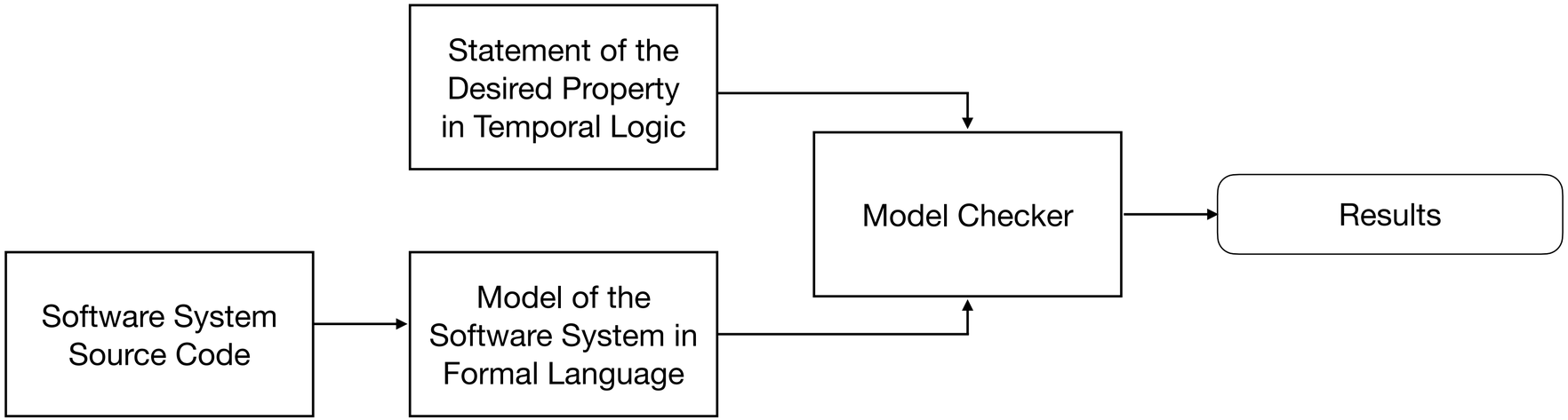


Model Checking

Model checking is an automated formal verification method that systematically explores the state space of a system model to verify whether specific properties hold. It is especially valuable for concurrent, reactive, or distributed systems, where traditional testing might overlook subtle bugs due to complex interactions.

It follows this workflow:

1. The system is described using a formal modeling language.
2. Some properties are specified in temporal logic, e.g., no deadlock occurs, or an API request is generated.
3. A model checker automatically explores all reachable states to verify the properties.
4. If a property is violated, the tool provides a counterexample trace demonstrating how the violation occurs.



Correctness by Construction

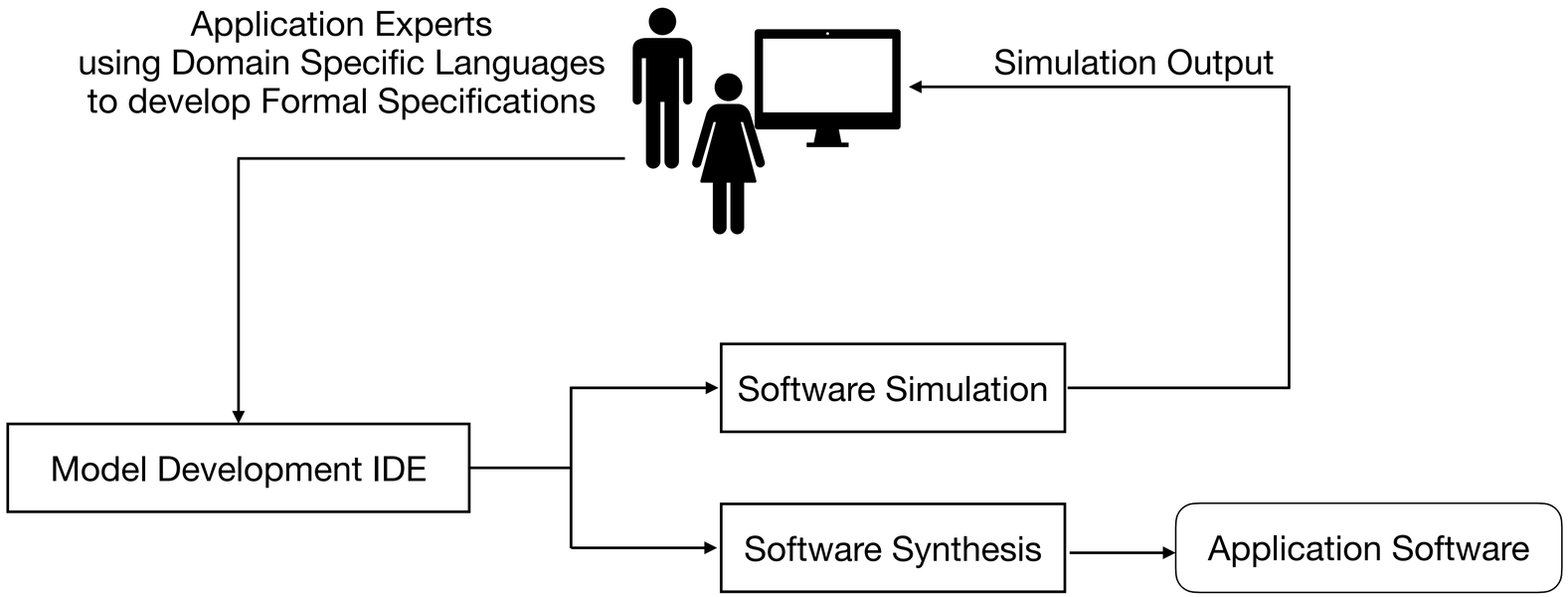
Humans are fallible!

Correctness by construction enables software development focused on avoiding and eliminating faults.

The goal is to ensure the implementation matches its specifications. It can be implemented in three ways:

- **Program Synthesis**: A program generator automatically generates the implementation in a high-level language from the specifications. Humans can guide the synthesis, or it can be fully automated. The main assumption is that synthesis introduces no defects. A specific example of program synthesis is model-based development.
- **Program Refinement**: The implementation is achieved by creating a series of transformations from a high-level abstract specification to a concrete implementation in a high-level language. The transformations and their proofs are generally manually developed. If the transformations and proofs are correct, the code is correct by design.
- **Program Analysis**: The implementation is carried out through manual development. However, each component is verified using program analysis techniques, e.g., static analysis, and components are built incrementally.

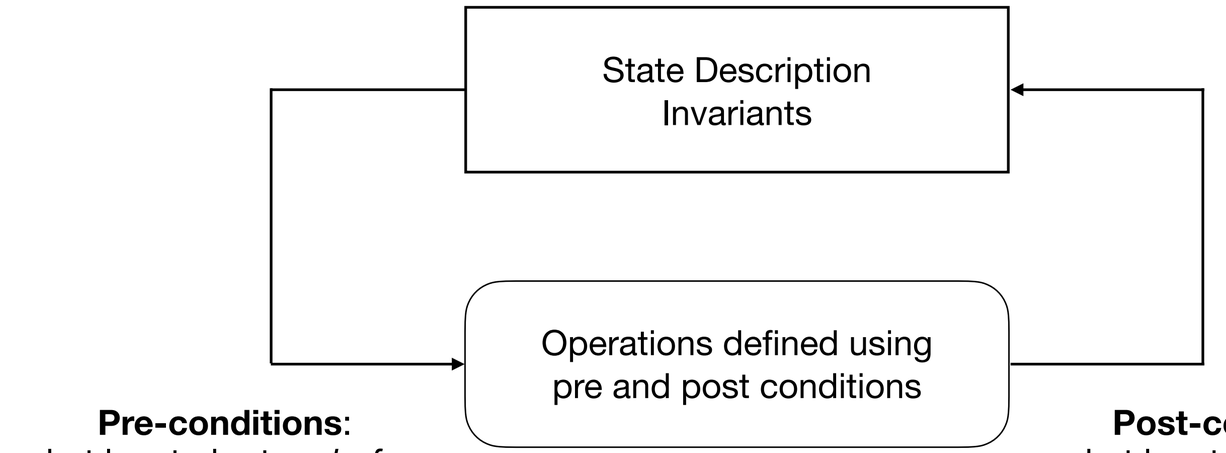
Model-Based Development



Advantages of Model-Based Development

- Less development effort
- Closer to the application, easier to validate
- Independent of programming language
- No bugs introduced by humans

Model-Based Specifications



State Description: definition of a set of variables and their related types
Invariants: predicates that must always be true for any implementation meeting the specifications

Developers use the specifications to precisely define what the software has to do.
All stakeholders can analyze them to have more confidence in the specifications.

Which document do you write to document these specifications?
Which formal language did you use?

Unified Modeling Language (UML) and Object Constraint Language (OCL)

The Unified Modeling Language (UML) and Object Constraint Language (OCL) allow you to

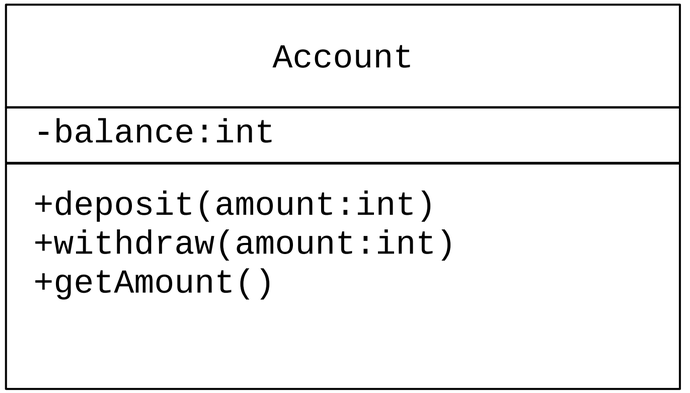
- Add information about the **visibility** of methods and attributes
  - Developers have different needs, and not all of them access the same methods and attributes of a class.
  - Visibility can be set to private (-), protected (#), or public (+).
  - Remember information hiding: show only what is necessary, and consider the trade-off with efficiency.
- Add information about **types** and **signatures**.
  - The attribute type indicates the range of permitted values and possible operations.
  - The signature provides similar details about the method parameters and their return values.
- Add **contracts**
  - Contracts enable developers to share information about classes. They include three types of constraints:
    - Invariants: Predicates that must always be true for all instances of a class or interface.
    - Preconditions: Predicates that must be true before a specific operation is invoked. They define constraints that a caller must meet before invoking an operation.
    - Postconditions: Predicates that must be true after an operation has been invoked. They specify constraints that the object must fulfill after the operation executes.

Example of UML and OCL

```
context Account
inv: self.balance >= 0

context Account::deposit(amount : Integer)
pre: amount > 0

context Account::deposit(amount : Integer)
post: self.balance = self.balance@pre + amount
```



OCL Pros and Cons

Pros

- Works well with UML models and supports model-driven engineering.
- Allows precise specification of invariants, preconditions, and postconditions.
- Purely declarative — ideal for expressing constraints without side effects.
- Supported by modeling tools like Eclipse OCL, Papyrus, and USE.
- Good for static analysis and model validation.

Cons

- Cannot access private elements unless they are exposed in the UML model.
- No support for side effects, state changes, or exception handling.
- Limited expressiveness for dynamic behavior, such as concurrency or events.
- Tool support is fragmented and inconsistent.
- UML has a steep learning curve for developers unfamiliar with it.
- Constraints have limited modularity and reusability.

Do we have alternatives?

Java Modeling Language (JML)

Java Modeling Language (JML) is a formal specification language for specifying the behavior of Java classes and interfaces using contracts. It allows developers to embed preconditions, postconditions, invariants, and other constraints directly in Java code as annotations.

Pros

- Designed for Java and integrates directly with source code.
- Supports comprehensive design-by-contract, including invariants, preconditions, postconditions, and exceptions.
- Allows specification of side effects such as assignable and pure functions.
- Supports exception handling through signals clauses.
- Offers strong tool support, like OpenJML, for runtime checking and formal verification.
- Easier for Java developers to learn.
- Provides better support for modular and reusable specifications.

Cons

- Limited to Java — not compatible with other languages or modeling tools.
- Needs access to source code, making it unsuitable for abstract modeling.
- Less ideal for early-stage modeling or platform-independent design.
- Might require annotations that clutter the code if not managed carefully.


Break